

Shipyard: A Multi-Leader Consensus Protocol with Auto-Balanced Leadership of the Sharded Keyspace

Xincheng Yang

Department of Computer Science
Illinois Tech
Chicago, IL, USA
xyang76@hawk.illinoistech.edu

Kyle C. Hale

School of Electrical Engineering and Computer Science
Oregon State University
Corvallis, OR, USA
kyle.hale@oregonstate.edu

Abstract—Leader-based consistency protocols often struggle with scalability due to the centralization of client request ordering. The leader, responsible for ensuring consistency by ordering reads and writes, becomes a bottleneck under high load, limiting overall system performance. This limitation is exacerbated by the tight coupling of consistency protocols with fault tolerance mechanisms, which further strain the leader’s capacity.

Keyspace sharding, a widely adopted technique in databases and blockchain systems, helps mitigate this bottleneck by distributing the workload. However, it introduces two primary challenges: first, maintaining consistency across shards, and second, balancing the workload between them. While several multi-leader approaches have been proposed to address consistency issues, they often neglect the crucial aspect of leadership balancing. Furthermore, these existing solutions frequently fail to consider dynamic factors such as CPU utilization, request frequency, and other performance metrics that significantly impact overall system efficiency.

To address these gaps, we propose Skiff, a variant of the Raft protocol that automatically balances leadership for a single shard. Furthermore, we introduce Shipyard, a higher-level protocol that manages all sharded keyspaces as a unified system, integrating failure recovery and offering linearizable reads and writes.

Index Terms—distributed systems, consensus, sharding, load balancing

I. INTRODUCTION

Maintaining consistency in a distributed system is a critical but challenging task. Leader-based consensus protocols like Paxos and Raft [1], [2] are commonly used to achieve consensus on the order of operations across nodes. These protocols rely on a leader to manage request ordering, ensuring a total order of writes for correctness and consistency. While this approach effectively maintains data integrity and ensures orderly processing, it introduces bottlenecks due to the leader’s centralized role. Specifically, the leader’s responsibility for handling all request ordering can result in performance degradation under high load. Consequently, these protocols face significant scalability challenges [3]–[6].

In a scenario with a *single* leader, many factors directly impact performance, the most significant being the leader’s CPU usage and network latency. Since all client requests must be processed and ordered by this leader, any increase in its workload can lead to bottlenecks. High CPU usage can slow down the processing of incoming requests, resulting in increased latency to handle client requests and reduced system throughput. Additionally, network latency can further exacerbate performance issues, as the leader must communicate with follower nodes to replicate requests and maintain consistency. Delays in these communications can lead to slower response times and limit scalability. Together, these factors highlight the weaknesses inherent in single-leader designs, underscoring the need for more resilient architectures that can effectively distribute load and limit single points of failure. For example, these single-leader protocols are often constrained to relatively small clusters, such as three, five, or seven nodes due to their scalability limitations.

To address the issues of a centralized single leader, multi-leader protocols enhance scalability and availability [7]–[10]. However, they also introduce two primary complexities that present challenges: conflict resolution and load balancing. Message complexity and fault tolerance also present challenges. Existing approaches may address some of these aspects but sacrifice others, generally falling into two categories: allowing all servers to propose requests [8], [9], [11] or sharding the keyspace and permitting only the shard leader to propose requests [7], [10], [12]–[14].

When all servers are allowed to propose requests, conflict resolution becomes challenging. Here, multiple servers can accept client requests simultaneously, leading to conflicting updates in the system. This requires a robust mechanism for reconciling these conflicts, introducing complexity and overhead to the protocol. As a result, rollback or additional communications are often necessary to resolve conflicts. For example, in EPaxos, an additional accept phase is introduced to resolve conflicts when requests interfere with one another [8]. This makes the protocol more complex and increases message size, as messages must encode dependency relation-

ships. Although it achieves load balancing and fault tolerance, EPaxos sacrifices efficiency in conflict resolution (due to added communications) and increases message complexity (by requiring dependency resolution).

As a different strategy, keyspace sharding [15] partitions data across multiple shards, each managed by a different shard leader. This approach aligns well with linearizability since it satisfies an important property called locality [16]. Locality ensures that two independently running, linearizably consistent systems can still maintain linearizability when considered as a whole. Thus, if we guarantee linearizability *within* each shard, the entire system remains linearizable. This reduces the need for additional conflict resolution. However, existing approaches often fall short in workload balancing and fault tolerance. For instance, Mencius uses a round-robin strategy to assign leadership across shards [7], but its approach can lead to load imbalance if the workload is unevenly distributed among shards, and the system can stall if a shard leader fails.

In this paper, we propose the Skiff protocol, a variant of Raft that introduces an additional parameter to automatically balance leadership. Like Raft, Skiff guarantees linearizability.

Additionally, we present a novel consistent keyspace sharding approach called Shipyard. Shipyard manages multiple Skiff instances, each corresponding to a single shard, and provides a unified mechanism for handling server failures. It decouples request ordering from fault recovery, avoiding redundant heartbeat messages between shards. Leveraging the locality property of linearizability and Skiff’s per-shard linearizability, Shipyard guarantees system-wide linearizability.

We make the following contributions:

- We propose **Skiff**, the first consensus protocol that uses workload as an electing factor, and provides automatic workload balancing functionality.
- We introduce the **Shipyard** approach, a multi-leader protocol that unifies and manages multiple Skiff instances to eliminate the single-leader bottleneck. Shipyard allows requests to be balanced and ordered based on their keyspace. It also decouples fault tolerance from request ordering, providing more robust consistency.
- We present a comprehensive evaluation of our approach, comparing against state-of-the-art methods.

Our code for Skiff and Shipyard is freely available and can be found at <https://github.com/xyang76/shipyard>.

The rest of the paper is structured as follows: Section II provides a detailed overview of consensus protocols and their associated scalability challenges. Additionally, it introduces keyspace sharding, along with its attendant problems. Section III introduces our design, with Skiff and Shipyard. Section IV presents a comprehensive evaluation of our approach, demonstrating its performance and scalability improvements. Section V concludes the paper and outlines future work.

II. BACKGROUND AND RELATED WORK

In this section we provide some brief background on replication systems and their challenges.

A. Replication Systems

Replication systems operate as a set of distributed processes (replicas) that replicate state to ensure correctness and reliability. They typically rely on an underlying state machine replication (SMR) protocol to maintain consistency across nodes despite failures and concurrency. Such SMR protocols are implemented using consensus algorithms like Paxos, Raft, and Zab [1], [2], [17], which address the challenge of reaching consensus in the presence of failures and partial synchrony. They employ various mechanisms to ensure that all replicas agree on the order of operations, which is crucial for maintaining a consistent state across the distributed system.

Paxos, a foundational SMR protocol, introduces three roles: proposer, acceptor, and learner [1]. It uses unique proposal numbers as logical clocks to reach consensus in partially synchronous settings. Paxos is a two-phase protocol with prepare and commit phases, requiring two round-trip times (RTTs). Raft simplifies the Paxos agreement using a log-based mechanism [2]. It defines three roles: leader, follower, and candidate, and introduces an incremental term number as its logical clock. Raft is also considered a two-RTT protocol, with leader election and log replication.

B. Challenges

Despite their effectiveness in maintaining consistency and fault tolerance, SMR protocols face significant challenges, particularly in scalability and efficiency under high load.

1) *Leader Bottleneck*: In leader-based SMR protocols, the leader handles all client request ordering and replication, creating a performance bottleneck. As the number of requests increases, the load on the leader grows, leading to delays and reduced throughput. This centralization means the leader must process all requests and ensure they are replicated to the follower nodes. During load spikes, the leader can become overwhelmed, causing slowdowns. If the leader fails, a new leader must be elected to allow forward progress, further degrading performance. Additionally, network communication is unbalanced, with the leader handling all traffic to and from follower nodes. This can cause network congestion around the leader, increasing latency and packet loss. The heavy load on the leader results in inefficient resource use, while follower nodes remain underutilized. In geographically distributed systems, these issues are more pronounced, affecting the leader’s ability to maintain timely request processing.

2) *Multi-Leader Protocols with Workload Balancing*: To address the limitations of single-leader SMR protocols, several multi-leader SMR protocols have been proposed. These protocols aim to eliminate the single point of failure and performance bottleneck associated with a leader by distributing coordination tasks among multiple nodes. However, very few approaches discuss workload balancing, and there is currently no automatic leadership balancing mechanism. EPaxos eliminates the need for a centralized leader by allowing any node to propose requests [8], [9]. It improves load distribution and fault tolerance, but handling conflicts adds coordination overhead. Non-conflicting requests commit in two RTTs, while

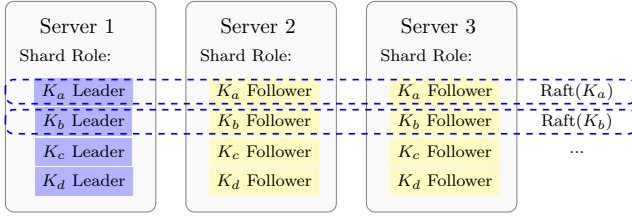


Fig. 1: The worst-case scenario of multiple Raft instances occurs when Server 1 is elected as the leader for all shards, while the other servers act as followers.

conflicting ones require three, increasing complexity and message size. Mencius distributes leadership in a round-robin manner across nodes [7], balancing load and avoiding single-node bottlenecks. However, it suffers from synchronization delays and struggles when workloads are unevenly distributed. DynaStar employs a replicated location oracle to dynamically adjust shards based on workload [18], improving multi-shard command handling and performance. However, reliance on the oracle risks bottlenecks, and its many message types increase message complexity.

3) *Other Multi-Leader Approaches*: Many other approaches involve sharding the entire keyspace (sometimes referred to as partitioning) and applying an internal consistency protocol within each shard. However, these approaches focus on consistency and lack mechanisms for load balancing. Slicer introduces sharding as a service, where the control plane that handles shard assignment is decoupled from the data plane, which handles request forwarding [19]. While Slicer realizes automatic sharding by dynamically balancing load, it does not integrate directly with the consensus layer. WanKeeper employs a hierarchical architecture along with a smart token migration mechanism to boost performance, ensuring object-level consistency [13]. However, WanKeeper introduces an additional token phase to guarantee consistency. ZooNet introduces additional client-side synchronization operations before cross-shard reads to coordinate requests, but this results in high coordination overhead [20].

4) *Multiple Raft Instances*: We can also run multiple Raft instances at the shard level. However, this may lead to an unbalanced workload, as shown in Figure 1. In Figure 1, each shard’s leader is randomly chosen if they have the same log sizes, which implies we cannot ensure who the actual leader is before the election. A worst-case scenario might be a single server elected as the leader for all shards. This is even worse than the single-leader case, as it requires more heartbeat information.

III. DESIGN

Our target is high-load coordination systems, especially those that are geo-replicated. These systems often struggle with performance bottlenecks stemming from the high volume of communication required between geographically dispersed nodes. We aim to address these challenges by optimizing request distribution and reducing coordination overhead, making

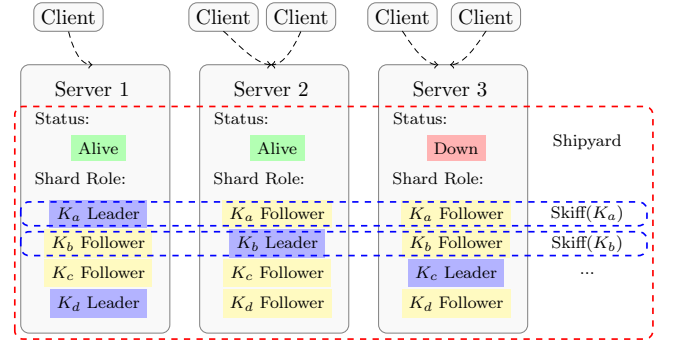


Fig. 2: System overview. We handle fault tolerance (status) and request ordering (shard roles) separately. Server 3, being in a down state, will revoke its leadership of the affected shards (Server 1 and Server 2 will run elections for K_c).

our design well-suited for systems where traditional methods may falter under stress. By focusing on these scenarios, we aim to enhance scalability, balance the workload, and improve overall system efficiency.

While exploring keyspace sharding across multiple shards, we observed that existing multi-leader protocols for sharding do not automatically adjust leadership to adapt to workload. To address this gap, we propose an approach that can automatically balance shard leadership. Furthermore, our design decouples request ordering from fault recovery in the SMR protocol, enabling multiple leaders to maintain consistency simultaneously.

A. System model

Multiple clients can concurrently submit requests to any of the replicated nodes and wait for a response. Client requests consist of *read* or *write* operations and can be processed synchronously or asynchronously. The *log* is an ordered sequence of entries that records these requests. Nodes can fail and later recover. A *peer* is a node that replicates logs. A *quorum* is a set of peers that make decisions to ensure consistency. We assume that the majority of the quorum will maintain liveness and reach consensus; otherwise, the system will halt until a majority is live again. All nodes are considered trusted, and we do not assume a Byzantine fault model. Although nodes are trustworthy, we assume their communication paths may become unstable. For example, two active nodes might not be able to connect with each other but could connect with a third node.

Our approach decouples liveness from request ordering, shown in Figure 2. Each node plays two roles: one for liveness (status) and another for request ordering (shard role). For request ordering, we shard the keyspace K , which represents the entire set of objects that clients can access. A *shard* or *partition* refers to a subset of these objects derived from the original keyspace K and denoted as K_p . To ensure consistency, we require that the sharded keys be *disjoint*. We use a *shard leader* to order all requests related to a single shard. In

State

All servers:

currentEpoch (persistent): The most recent epoch number encountered by the server, starting at 0 and updated in a monotonically increasing order.

currentApportion (persistent): The highest apportion accepted during the current epoch. Within the same epoch, the apportion increases monotonically.

log[] (persistent): Log entries, where each entry contains a command along with its associated epoch and apportion.

commitIndex (volatile): The index of the most recent committed log entry known by the server.

lastApplied (volatile): The index of the most recent log entry applied by the server.

proposeApportion (volatile): The proposed apportion, which is periodically updated by the follower itself to reflect its system conditions.

Leaders:

peerIndex[] (volatile): For each peer server, this is the index of the highest log entry that is known to be committed.

grantedApportion (volatile): A number used to facilitate re-balancing, prevent concurrent elections, and ensure valid election processes. It is initialized to 0 and reset periodically.

VoteAndGather RPC

Arguments:

epoch: The candidate's current epoch.

apportion: The candidate's apportion.

candidateCommit: The index of the candidate's most recently committed log entry.

Results:

epoch: The receiver's current epoch.

apportion: The receiver's current apportion.

peerCommit: The index of the receiver's most recently committed log entry.

diff[]: Log entries starting from the sender's commitIndex.

voteGranted: **true** if the vote is granted by the receiver.

Receiver implementation:

1. If the sender's candidateCommit $\leq$ the receiver's commitIndex, reply with the diff entries from the candidate's commit index, and peerCommit = receiver's commitIndex.
2. Grant vote if epoch $>$ currentEpoch or (epoch = currentEpoch and apportion $>$ currentApportion).

AppendEntries RPC

Arguments:

epoch: The leader's current epoch.

apportion: The leader's apportion.

leaderId: Allows the follower to redirect clients.

startIndex: The starting index for the new log entries.

diff[]: Log entries to store (empty for heartbeat messages).

leaderCommit: The leader's current commit index.

Results:

epoch: The receiver's current epoch.

apportion: The receiver's apportion.

followerCommit: The index of the receiver's most recently committed log entry.

ok: **true** if the append entries are successful.

Receiver implementation:

1. If epoch $>$ currentEpoch or (epoch = currentEpoch and apportion $\geq$ currentApportion), accept the entries and respond with *ok* = true, otherwise, *ok* = false.
2. If accepted, respond *followerCommit* as the index of the last newly appended entry.
3. If accepted, update *commitIndex* to max(leaderCommit, commitIndex).
4. If accepted (leader still valid) and apportion $<$ proposeApportion, send Balance RPCs to all other servers.

Balance RPC

Arguments:

leaderId: The unbalanced leader.

proposalNum: The apportion proposed by the follower.

Results:

tokenGranted: True if the unbalanced leader grants the token, otherwise false (to avoid concurrent elections).

ok: True if the server successfully received the Balance RPC (to ensure a majority of servers are connected).

Receiver implementation:

1. Set *ok* = true in any state.
2. If the receiver is a leader and the leaderId matches, and if grantedApportion $<$ proposalNum, set grantedApportion = proposalNum and reply with tokenGranted as true. Otherwise, reply tokenGranted as false. Reset the grantedApportion = 0 after a timeout.

Fig. 3: A condensed summary of the Skiff protocol (part 1).

Figure 2, a key space K is sharded into parts a through d . Each sharded key runs a Skiff instance to automatically balance the workload.

For liveness, we introduce Shipyard, a unified liveness mechanism that handles failures across the system. This mechanism uses a unified timeout to monitor node heartbeats and detect unresponsive nodes. Upon detecting a failure, it coordinates recovery for all shards, ensuring efficient leadership reassignment and maintaining overall system stability. For example, in Figure 2, Server 3 will revoke the leadership of its managed shards, and another node will automatically elect a new leader based on Skiff.

B. Skiff

Skiff is a variant of Raft designed to elect a single leader per shard and achieve load balancing. Skiff uses three roles: *leader*, *follower*, and *candidate* for quorum peers, similar to Raft. Skiff ensures that each shard has at most one leader at any given time. In this log-based algorithm, the leader is responsible for replicating logs to the followers until it loses leadership. Skiff has three phases: *VoteAndGather*, *AppendEntry*, and *Balance*. We modified the election condition in Raft’s *RequestVote* phase so that the *Balance* phase can initiate a re-election if the system becomes unbalanced. Thus, we introduce an extra parameter, *apportion*, to balance the workload distribution for the winner. In Skiff, each node stores a current epoch number and a current *apportion* as a pair, acting as a logical clock to determine order and ensure correctness. We represent *apportion*, the key factor used to balance the workload, as a scalar value that can be derived from multiple load signals. Which of these signals is chosen is up to the protocol implementation. For example, we can use request frequency, CPU usage, or the number of shards that each node manages to achieve balanced workloads between shards. We present a condensed summary of Skiff in the style of the Raft paper [2] in Figures 3 and 4. Although the algorithm superficially resembles Raft, its correctness is actually more aligned with Paxos. We can consider the pair $\langle \text{epoch}, \text{apportion} \rangle$ as Paxos’s proposed number, while the logs from zero to the highest committed logs represent Paxos’s highest chosen value. Meanwhile, the *VoteAndGather* phase can be considered as Paxos’s phase 1, and *AppendEntries* can be considered as Paxos’s phase 2. Skiff uses the pair $\langle \text{epoch}, \text{apportion} \rangle$ to divide time and ensure a monotonically increasing order. Epoch increases with consecutive integers, while *apportion* is proposed based on workload. The epoch has higher priority than *apportion* during comparison. In the algorithm, two leaders may have the same epoch, but since we ensure a monotonically increasing sequence for the pair, the higher *apportion* is the chosen leader of the majority. Each pair starts with an election, in which one or more candidates attempt to lead and commit logs. The election may fail, resulting in no leader during the current pair’s period, and a new election will increase the epoch and restart shortly.

The tuple consisting of $\langle \text{epoch}, \text{apportion} \rangle$ is exchanged during node communications, which we refer to as a *term*.

Rules for Servers
All servers: • If $\text{commitIndex} > \text{lastApplied}$, increment lastApplied to commitIndex and apply logs from lastApplied to commitIndex to the state machine. • If any RPC message has $\text{epoch} > \text{currentEpoch}$ or ($\text{epoch} = \text{currentEpoch}$ and $\text{apportion} > \text{currentApportion}$), become a follower. Followers: • If the follower does not receive <i>AppendEntries</i> from the current leader or does not grant a vote to a candidate after a timeout, become a candidate. • If an <i>AppendEntries</i> RPC is accepted and $\text{apportion} < \text{proposeApportion}$, send <i>Balance</i> RPCs to all other servers. • If a <i>Balance</i> RPC is sent and a majority responds with <i>Balance</i> RPCs and the token is granted by the current leader, become a candidate. Candidates: • A candidate starts an election and will restart a new election if the election timeout occurs: • Increment currentEpoch by one and set $\text{currentApportion} = \text{proposeApportion}$. • Add one to the votes received (vote for itself). • Reset the election timer. • Preserve its commitIndex. • Broadcast <i>VoteAndGather</i> RPCs. • If a <i>VoteAndGather</i> RPC is received and $\text{peerCommit} > \text{candidate's commitIndex}$: • Append logs starting from the preserved commitIndex. • Update $\text{commitIndex} = \text{peerCommit}$. • Update $\text{peerIndex}([\text{peer}]) = \text{peerCommit}$. • If a <i>VoteAndGather</i> RPC is received and $\text{commitIndex} = \text{candidate's commitIndex}$: • Choose the longest log entries starting from commitIndex. • Update $\text{peerIndex}([\text{peer}]) = \text{peerCommit}$. • If an <i>AppendEntries</i> RPC is received during the election with a higher term, become a follower. • Become the leader if a majority of votes are received. Leaders: • Send an initial <i>AppendEntries</i> RPC after the election. • Periodically send empty <i>AppendEntries</i> (heartbeat) to each server. • If a client’s command is received, append the entry to local logs and respond if the majority is committed. • If the leader’s latest log index $\geq \text{peerIndex}$ for a follower, send an <i>AppendEntries</i> RPC with the diff log entries starting at peerIndex. Update peerIndex based on the replied followerCommit if the append entries for that follower are successful. • For all indexes in peerIndex, if N is the highest committed index of the majority, set $\text{commitIndex} = N$. • If proposeApportion changes, become a candidate (vote for the new pair of epoch and <i>apportion</i>).

Fig. 4: A condensed summary of the Skiff protocol (part 2).

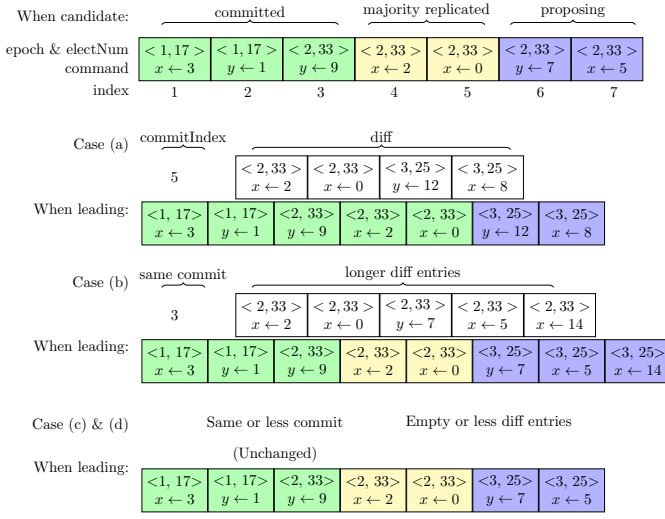


Fig. 5: When a candidate is elected as the leader, it gathers logs from the majority. There are four cases: the first two cases will update its logs because the peers have the latest leader information, while the last two cases will not update its logs. If a log is majority replicated, when the leader receives a majority response, that log will be gathered from at least one peer.

We keep updating this tuple $\langle \text{epoch}, \text{apportion} \rangle$ and select the higher one. The node rejects requests that have a lower $\langle \text{epoch}, \text{apportion} \rangle$. A candidate or a leader will convert to the follower state if it receives a higher pair of $\langle \text{epoch}, \text{apportion} \rangle$.

Skiff uses remote procedure calls (RPCs) to communicate with other nodes. There are three RPCs in the protocol: `VoteAndGather`, `AppendEntries`, and `Balance`. The `VoteAndGather` RPC is initiated by the candidate to elect a new leader and ensure that the leader holds the highest committed log by majority. `AppendEntries` is used for replicating logs, and is initiated by the leader. The `Balance` RPC is invoked by a follower to prepare for re-election if the current environment is unbalanced.

1) *Leader Election*: During leader election, we perform two tasks: voting for the leader and ensuring that the elected leader has the highest replicated log from the majority. We focus first on the voting process.

The Skiff node starts in the follower state and will transition to the candidate state if no heartbeats are received from the leader after a timeout period. The candidate will initiate an election by increasing its current epoch and choosing an apportion as a pair to propose as its term for voting. Additionally, it will vote for itself and send `VoteAndGather` requests to all other nodes. The candidate continues until (a) it wins the election, (b) it is notified that another node with a higher term pair exists, or (c) no winner is declared within the given time period. The first two cases are similar to Raft: (a) ensures that only one leader exists for the given term pair, and (b) ensures that the leader has the highest term and is eligible to append entries to the replicated logs. In the third case, no

winner is declared within the given time period, indicating that many followers became candidates simultaneously, and none of them received a majority vote. This leads to a new election. However, if the epoch and apportion remain the same, this could result in a split vote that repeats indefinitely. We use apportion randomization to avoid this scenario, as discussed in Section III-B4. The randomized apportion reduces the likelihood of another split vote.

We also ensure that the elected leader has the highest replicated log and commit index from the majority. We guarantee the safety of log replication for logs before the commit index. Figure 5 illustrates an example of a candidate being elected leader. A log can be divided into three parts: committed, majority replicated, and proposing (not yet replicated by the majority and potentially lost if the leader crashes). We gather logs from the majority with four possibilities, as shown in Figure 5: (a) the replied commit index (from the peer) is greater than the candidate's commit index, along with a set of differing log entries; (b) the replied commit index equals the candidate's commit index, and the differing log entries are longer than the current logs; (c) the replied commit index equals the candidate's commit index, and the differing log entries are shorter than the current logs; (d) the replied commit index is less than the candidate's commit index, with an empty set of log entries.

When the replied commit index is higher than the candidate's commit index, we set the candidate's commit index to match the replied index and overwrite any uncommitted log entries with the differing entries received from the peer. This indicates that the peer has the most recent communication with the old leader, allowing us to directly synchronize the log entries from the peer and update the peer's commit index for future append log operations. If the replied commit index equals the candidate's commit index, we select the longest log entries, as they reflect the latest communication with the old leader, and subsequently update the peer's commit index. Finally, if the replied commit index is less than the candidate's commit index, we only update the peer's commit index without making any other changes.

After the new leader gathers logs from the majority, it must ensure that it has the highest majority replicated logs. Meanwhile, when the new leader is elected, there are two possibilities for its commit index: (a) the new leader's commit index matches the old leader's, or (b) the new leader's commit index is less than the old leader's index. In the first case, the new leader's commit index is globally the highest. If this case occurs, it means the old leader has not updated the commit index to reflect the majority during this time period. The new leader can thus safely continue appending logs from its commit index.

In the second case, we will demonstrate that the old leader's committed log will also be committed by the new leader. Since the old leader's commit index is updated after majority replication (as guaranteed by the commit index update policy in Section III-B2), if a log entry is committed by the old leader, it implies that the log entry has already been replicated by

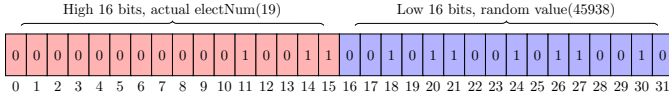


Fig. 6: A 32-bit apportion; the first 16 bits represent the actual apportion value (e.g., 19), which takes precedence in comparisons, while the last 16 bits are a random tiebreaker value (e.g., 45938).

the majority. Thus, the new leader, gathering logs from the majority, will ensure that this log entry is collected at the same log index. Any log committed by the old leader will thus be safely committed by the new leader. As a result, we can confidently apply the logs prior to the commit index to the state machine.

2) *Log Replication*: When a leader has been elected by a majority, it will begin appending logs to followers. The leader will first append the request as a log entry to its own logs and then issue `AppendEntries` RPCs to other nodes. When the log has been replicated to a majority, it will apply log entries to its state machine and update the commit index. This phase can directly reuse Raft’s log replication mechanism, as we will detail in Section III-B5. For brevity, we simplify the description by using committed log indices.

In normal operation, the leader will remain active and broadcast `AppendEntries` RPCs to followers, ensuring that log replication is consistent. However, there may be inconsistencies with uncommitted logs. For example, an old leader may have been elected and proposed some entries that were not replicated by the majority. Therefore, we begin appending entries from the latest reported commit index of the followers. After the election, the leader gathers the latest commit indices from the majority and overwrites the followers’ logs starting from their commit indices. The leader sends the follower the differing entries from its commit index. The follower will accept this if no message from a leader with a higher term pair arrives. Once accepted, the follower overwrites its logs for the uncommitted index to maintain consistency with the leader, updates its commit index to match the leader’s commit index, and reports its current log index as the leader’s commit index. At this point, from the leader’s perspective, the follower’s commit index includes the leader’s proposed entries; however, from the follower’s perspective, its actual commit index is the latest commit index from the leader.

The leader will update the commit index if the majority of followers acknowledge that index. This ensures the leader’s commit index reflects the success of the majority replication. Since all followers’ commit indices are updated based on the leader’s broadcasts, their commit indices are also guaranteed to be updated after replication is successful.

3) *Load Balancing*: When a follower receives `AppendEntries` RPCs from the leader, it compares the leader’s apportion with its own proposal apportion. If the follower has a higher apportion than the leader, it can initiate

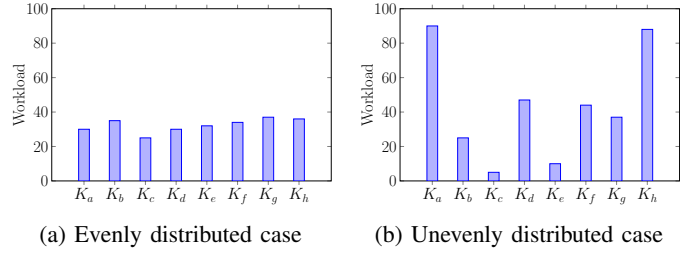


Fig. 7: Workload distribution across shards. The workload is highly dependent on how the user shards the data.

a re-election.¹ However, we must address two issues first: (a) the network connectivity for the follower, and (b) the possibility of concurrent re-elections from multiple followers with apportion values higher than the leader.

We introduced the Balance phase to address these two issues. For the first issue, a follower will send `Balance` RPCs to all other nodes, and any node that receives it will reply to confirm connectivity. To tackle the second issue, we need the leader to reduce the concurrency of re-elections. We use an integer `grantedApportion` in the leader to restrict concurrent elections: a follower can only initiate a re-election if its proposed apportion is higher than the leader’s `grantedApportion` value; otherwise, the leader will not grant the follower permission to start the election. `grantedApportion` will be reset to zero after a specified time period. Followers with a lower apportion may initiate an election after the leader’s reset, but this neither affects correctness nor has a significant impact on concurrent re-elections. If a follower receives a majority response to its `Balance` RPCs and a grant from the leader, it will start a re-election. Since it increases the epoch value and its proposed apportion is higher than the current leader’s, it will have a greater chance of winning the leader election.

4) Choosing and Changing apportion:

Setting apportion: Skiff will choose voting winners based on higher apportion values. In some cases, however, apportion may be derived from metrics where lower values have preference, such as a node with lower CPU usage being the election winner. In this case, the implementation can simply invert the relationship, e.g., with XOR or negation.

Additionally, if multiple candidates concurrently run elections, as described in Section III-B1, split voting may lead to a livelock where no leader is chosen. To address this issue, we can combine the apportion with a random value. For example, in a 32-bit integer representing the apportion, the high 16 bits can be used for the actual apportion and the low 16 bits for a random value, as shown in Figure 6.

Changing apportion: The apportion is part of the term pair and is involved in log replication, so we cannot arbitrarily change it. We only allow the leader to change its apportion,

¹If we use system load to determine apportion, then we would set the apportion value inversely proportional to its load, e.g., low load→high apportion).

and if it does so, it must also increment its epoch number to ensure that the new pair is always greater than the old pair. For a leader that may need to frequently update its apportion, an optimized version is in Section III-B5 for dynamic updates and checking.

5) Variants and Optimization:

Append Entry from Latest Entries: In the Skiff protocol, we start appending entries from the committed index. This simplifies our protocol description but may lead to frequent overwriting of existing log entries. To avoid this issue, we can adopt Raft’s log replication mechanism by introducing the nextIndex and checking not majority replicated logs. We omit the detailed algorithm here, as the AppendEntries phase can directly use Raft’s AppendEntries mechanism.

Threshold for balancing: The balance phase may encounter a “swap” situation. For example, consider two nodes, S1 and S2, each consuming 50 percent of the workload for other tasks. If one of the nodes wins the election and becomes the leader, it will handle an additional 20 percent of the load. In this scenario, the leader and follower could continually swap roles, leading to an unstable ‘ping-pong’ effect. To prevent this, we can introduce a threshold. If the leader’s proposed apportion exceeds the current apportion plus the threshold, it can re-initiate the election.

C. Shipyard

Previously, we discussed heartbeats and timeouts for detecting failures in Skiff to ensure single-shard fault tolerance. However, in a multi-leader setup, if each shard independently uses heartbeats and timeouts to detect failures, this can result in a large number of redundant messages. To address this, we introduce Shipyard, a unified protocol that manages all heartbeat and failure information at a global level. Additionally, within Shipyard, each message includes extra information (specifically, the shard name) to distinguish it from other shards. Shipyard has two main components: sending heartbeats and detecting heartbeats. When Shipyard starts, it initializes two timers, leaderTimer and shardTimer, which are responsible for sending and detecting heartbeats, respectively. We provide a condensed summary of the Shipyard algorithm in Figure 8.

Shipyard introduces HeartBeats RPCs, which are used to maintain heartbeat information between nodes. When a node is elected as the leader for any shard, it periodically sends HeartBeats RPCs to other nodes, including all of its leading shards’ information. When a node receives a HeartBeats RPC, it refreshes the timeout for the corresponding shards based on the sender’s leading shards. If any shard times out, Skiff elections are triggered for that shard. Additionally, whenever any Skiff RPC is received, we also update the corresponding timeout for that shard, ensuring that any message from another node refreshes the timeout interval.

Fault Tolerance in Shipyard: Shipyard ensures $(2f + 1)$ fault tolerance at the shard level, where f represents the maximum number of node failures that can be tolerated. For example, if Skiff for that shard is deployed with three nodes, a maximum of one node can fail. A typical deployment consists

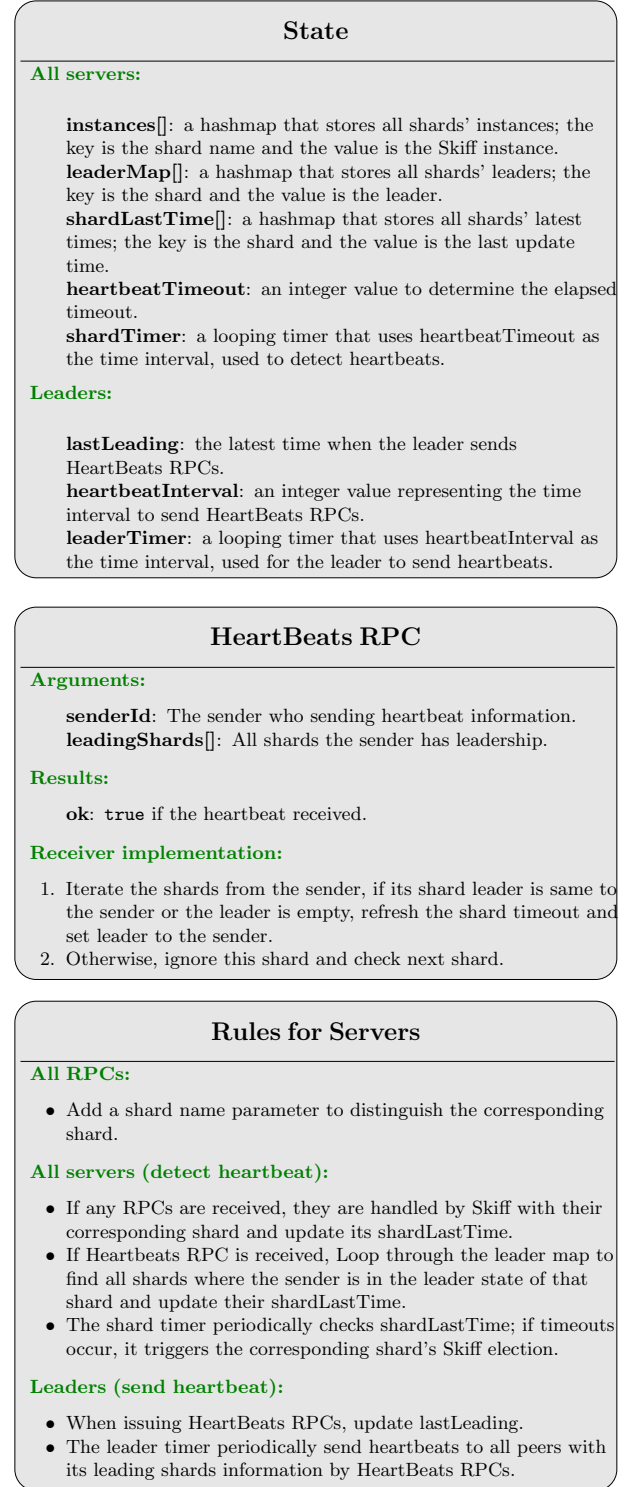


Fig. 8: The Shipyard consensus protocol. Shipyard decouples heartbeats from the shard-level protocol and handles node liveness independently.

of an odd number of nodes, such as 3, 5, or 7. This deployment can also be scaled to more nodes when needed.

Scaling: In Figure 2, we illustrate a scenario with three nodes and four shards, where all nodes maintain all shards across all keyspaces divided from K . However, in our protocol, a node may or may not manage all shards. When scaling occurs, a node may manage only some shards. For example, in a case with 50 shards and 30 nodes, if the shard workload is high, we can ensure that each shard is replicated on a minimum of five nodes to achieve two-fault tolerance. In this case, a node need not maintain information for all 50 shards, and the minimum number of shards managed would be calculated as $5 \times \frac{50}{30} \approx 8.33$, which can be rounded to 9.

IV. EVALUATION

We compared Shipyard with Raft, EPaxos, Mencius, and Multi-Raft. Multi-Raft refers to the classic Raft protocol but using multiple shard instances (Figure 1). For Raft and Multi-Raft, we implemented our own version of the protocol in Go version 1.22 using the same I/O strategies as outlined in the original protocol description. We use the EPaxos authors’ implementations for EPaxos and Mencius directly. We selected two types of apportion strategies: one based on request frequency (sy-frequency), and the other based on the number of shards currently managed by a leader (sy-managed). We are looking to answer the following questions:

- How does throughput compare for these approaches near saturation?
- How is the workload balanced in a non-saturated environment?
- How do these approaches perform in a failure-prone environment?

Experimental Setup: Our setup uses Amazon AWS EC2 with three data centers. We deployed two servers in the US East region (North Virginia), two servers in Asia Pacific (Tokyo), and one server in Europe West (Paris). Additionally, we deployed five instances (two in the US, two in Japan, and one in Europe) as clients to access the servers. Each instance ran three clients independently. All instances used the t3.micro type, configured with two vCPUs and 1GB of RAM. Each instance has 8 GB of gp3 SSD storage, supporting 10 GB/s cached read speeds and 150 MB/s buffered disk reads. The internal network bandwidth within each data center was 1 GB/s, with a latency of 0.5 ms. The network bandwidth between servers was as follows: Japan and US (10 MB/s, with 150 ms latency), EU and Japan (2 MB/s, with 210 ms latency), and US and EU (4 MB/s, with 80 ms latency).

We evaluate two types of workloads: evenly sharded and unevenly sharded. Client requests are divided into 9 shards. For the unevenly distributed shards, we use the letter-frequency distribution derived from an English corpus as the key. To increase imbalance, we sort the frequencies and present the frequency of letters in each shard, as shown in Table I. Unless otherwise noted, we repeat each experiment for 10 trials and report the arithmetic mean, and run experiments for roughly 100 seconds.

Shard Name	Letters	Frequency
Shard 1	E, T, A	12,702, 9,056, 8,167
Shard 2	O, I, N	7,507, 6,966, 6,749
Shard 3	S, H, R	6,327, 6,094, 5,987
Shard 4	L, D, C	4,025, 4,253, 2,782
Shard 5	U, M, W	2,758, 2,406, 2,360
Shard 6	F, G, Y	2,228, 2,015, 1,974
Shard 7	P, B, V	1,929, 1,492, 978
Shard 8	K, X, J	772, 150, 153
Shard 9	Q, Z	95, 74

TABLE I: Shard names, letters, and frequencies.

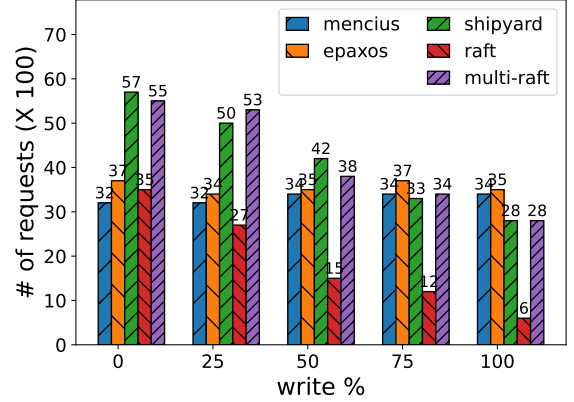


Fig. 9: Throughput of 15 clients with saturated environment, measured for 3 seconds.

Request throughput: We first evaluate throughput in a fully saturated environment by simulating 15 clients generating requests simultaneously. The results are shown in Figure 9. We observe that the performance of EPaxos and Mencius remains almost constant as the write percentage changes. This is because both protocols replicate read operations to all replicas to guarantee consistency. In contrast, the performance of Raft, Multi-Raft, and Shipyard gradually degrades as the write ratio increases, since read requests are handled by the leader rather than being replicated to all replicas. As a result, Shipyard and Multi-Raft achieve up to 1.5× higher performance than EPaxos and Mencius. We also observed heavier load and more frequent load spikes as the write ratio increases. Under these conditions, EPaxos and Mencius outperform Multi-Raft and Shipyard by 1.2× since EPaxos and Mencius do not shard the keyspace, whereas Shipyard and Multi-Raft incur additional contention on communication channels multiplexed between shards. Finally, Raft performs the worst in all cases due to its single-leader bottleneck. In this saturated environment, Shipyard does not exhibit a significant advantage over the other approaches, since all replicas are so heavily loaded by incoming requests that there is not sufficient imbalance between replicas to capitalize on. In the next section, we will see how as imbalance arises in less fully saturated environments, Shipyard starts to show benefits.

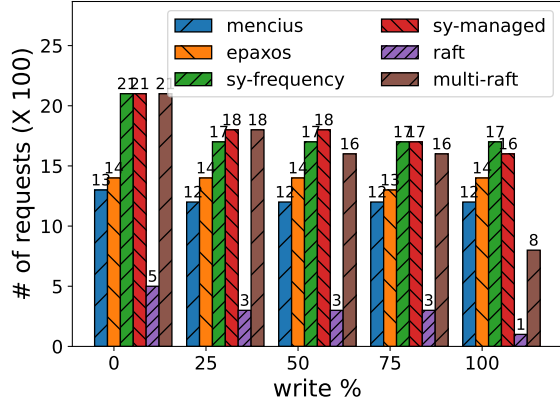


Fig. 10: Throughput when request rates are limited under an evenly distributed workload.

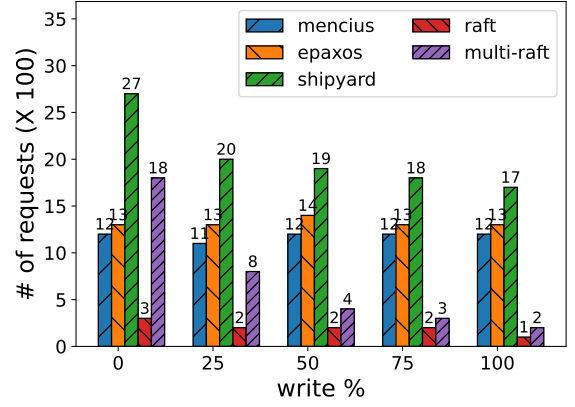


Fig. 12: Throughput when there are fewer shards (3) than servers (5).

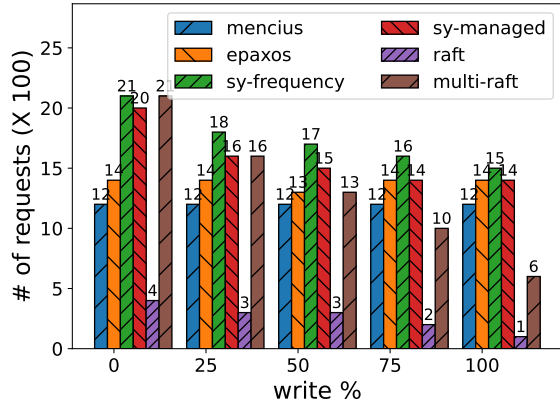


Fig. 11: Throughput when request rates are limited under unevenly distributed workloads.

Load balancing: We evaluate performance in a non-saturated environment by limiting the request rate to 100 per round to limit server congestion and saturation. The workloads are distributed under both even and imbalanced scenarios, where request frequencies follow the letter-frequency distribution of an English corpus, as listed in Table I.

We first evaluated throughput for balanced workloads. Figure 10 shows our results. The average CPU usage during these tests ranged from 60 percent to 80 percent. Among all approaches, Raft demonstrated the lowest performance due to its heavy reliance on a single leader. Multi-Raft, with its randomly assigned leaders, does not prioritize load balancing. In contrast, Shipyard, which distributes leaders based on request frequency and balances load more effectively compared to unbalanced approaches outperformed both Raft and Multi-Raft. Shipyard achieves 1.3 $\times$ better performance than EPaxos and Mencius on pure writes, since EPaxos requires an additional conflict resolution phase and Mencius needs a round-robin mechanism to handle requests.

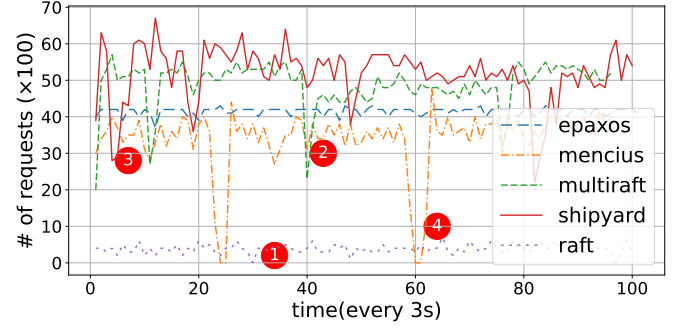


Fig. 13: Failure and recovery.

We also evaluated throughput for imbalanced workloads, with the results shown in Figure 11. Under these conditions, Raft and Multi-Raft performed similarly to their results in evenly distributed workloads, as neither can adjust based on load. Among the Shipyard approaches, Shipyard-frequency maintained a better load balance despite the uneven request distribution. However, Shipyard-managed failed to achieve effective load balancing, leading to degraded performance under imbalanced workloads. We observed that the write performance of Multi-Raft shows relatively high variation due to the randomness in leadership assignment.

Failure and Recovery: While the above configuration is based on having more shards (9 shards) than servers (5 servers), we also conducted an evaluation where the number of shards was fewer than the number of servers. Specifically, we deployed 3 shards across 5 servers. The results are shown in Figure 12. We see that Shipyard achieves higher throughput due to reduced channel contention between shards. However, for the Multi-Raft version, since it does not balance load automatically, one server was assigned as the leader for two shards. As a result, its performance is approximately 0.7 $\times$ -0.5 $\times$ that of Shipyard for reads. Moreover, we observed that in both Raft and Multi-Raft, updating the commit index during `AppendEntries` incurs an $O(n)$ cost per call, because

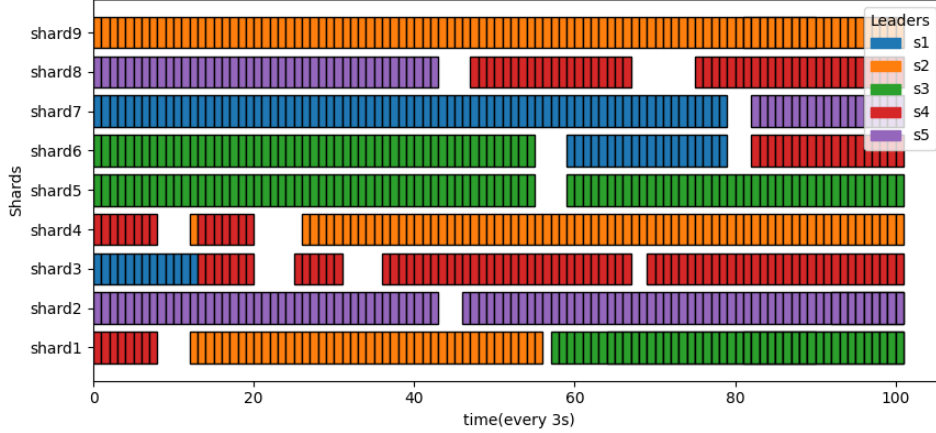


Fig. 14: Shard leadership changes.

the protocol must check the term of each entry to avoid inconsistencies. This substantially slows down performance in this scenario with fewer shards. In contrast, our protocol’s commit-index-based log append is an $O(1)$ operation and therefore does not suffer the same performance degradation.

We also conducted a failure and recovery evaluation on the above approaches, as shown in Figure 13. We randomly trigger server crashes and recoveries. Based on our protocol, each server starts as a candidate and will initiate a leader election if it does not receive leader heartbeats within a given period. Multiple servers can crash simultaneously, leading to system unavailability. The evaluation was performed with 20 percent write operations across 9 shards. We ran this experiment for 300 seconds.

From Figure 13, we see that for single server crash and recovery ①, the multiple-leader approaches (Multi-Raft, Shipyard) exhibit higher availability than the single-leader scenario (Raft). This is because other shard leaders remain available, unlike in the single-leader case where a crash causes total unavailability.

Additionally, we observe the recovery periods for Shipyard and Multi-Raft in ② and ③. Since Shipyard requires an extra rebalancing phase, it experiences a longer recovery time compared to Multi-Raft after ③. However, despite this additional recovery time, the rebalancing process improves Shipyard’s overall performance compared to Multi-Raft. Shipyard maintains relatively stable performance, whereas Multi-Raft is highly dependent on leader placement. For example, Multi-Raft shows performance improvements at the beginning and end of the evaluation because its randomly selected leaders were more balanced during those periods. We also observed that EPaxos experiences almost no performance degradation under a single-server failure, since other servers can continue processing requests normally. In contrast, Mencius suffers a prolonged period of unavailability when a single server fails, as shown at ④.

We also measured shard leadership balance under failure

and recovery, as shown in Figure 14. Here we see that each server generally leads only two shards as a result of the balancing strategy. Occasionally, due to random server crashes, a server may lead up to three shards at a given time. This demonstrates that our balancing strategy is correctly applied in these scenarios.

V. CONCLUSIONS AND FUTURE WORK

We presented Skiff, the first auto-balancing protocol that achieves balance by swapping leadership. It introduces an additional parameter to determine whether the system is balanced. Additionally, we introduce Shipyard, a protocol that manages multiple Skiff instances and provides unified failure handling. We ensure linearizability based on the locality property, guaranteeing that each shard is linearizable.

There are several opportunities for future work. First, we do not allow local reads, as enabling local reads only satisfies sequential consistency and breaks linearizable consistency [21], [22]. However, we could enable local reads by introducing other mechanisms to guarantee a weaker consistency for geo-replicated systems. Secondly, we also plan to enable automatic swapping of objects between shards in Shipyard if the user’s sharding approach is not evenly balanced [23]–[25]. This can be achieved with appropriate delete and insert operations.

REFERENCES

- [1] L. Lamport, “Paxos made simple,” *SIGACT News*, vol. 32, no. 4, pp. 51–58, 2001.
- [2] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proceedings of the USENIX Annual Technical Conference*, ser. USENIX ATC ’14, Jun. 2014, pp. 305–319.
- [3] C. Tang, T. Kooburat, P. Venkatachalam, A. Chander, Z. Wen, A. Narayanan, P. Dowell, and R. Karl, “Holistic configuration management at facebook,” in *Proceedings of the 25th Symposium on Operating Systems Principles*, ser. SOSP ’15, Oct. 2015, pp. 328–343.
- [4] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the 6th International Workshop on Networking Meets Databases*, ser. NetDB ’11, vol. 11, Jun. 2011, pp. 1–7.

- [5] A. Ailijiang, A. Charapko, M. Demirbas, B. O. Turkan, and T. Kosar, "Efficient distributed coordination at WAN-scale," in *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems*, ser. ICDCS '17. IEEE, Jun. 2017, pp. 1575–1585.
- [6] A. Ailijiang, A. Charapko, M. Demirbas, and T. Kosar, "WPaxos: Wide area network flexible consensus," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 1, pp. 211–223, 2019.
- [7] Y. Mao, F. P. Junqueira, and K. Marzullo, "Mencius: building efficient replicated state machines for WANs," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI '08. USENIX Association, Dec. 2008, p. 369–384.
- [8] I. Moraru, D. G. Andersen, and M. Kaminsky, "There is more consensus in egalitarian parliaments," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, ser. SOSP '13, Nov. 2013, pp. 358–372.
- [9] S. Tollman, S. J. Park, and J. Ousterhout, "EPaxos revisited," in *Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI '21, Apr. 2021, pp. 613–632.
- [10] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, "CockroachDB: The resilient geo-distributed SQL database," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '20. New York, NY, USA: Association for Computing Machinery, May 2020, p. 1493–1509. [Online]. Available: <https://doi.org/10.1145/3318464.3386134>
- [11] P. Tennage, C. Basescu, L. Kokoris-Kogias, E. Syta, P. Jovanovic, V. Estrada-Galinanes, and B. Ford, "QuePaxa: Escaping the tyranny of timeouts in consensus," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23, Oct. 2023, pp. 281–297.
- [12] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaure, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems*, vol. 31, no. 3, Aug. 2013. [Online]. Available: <https://doi.org/10.1145/2491245>
- [13] A. Ailijiang, A. Charapko, M. Demirbas, B. O. Turkan, and T. Kosar, "Efficient distributed coordination at WAN-Scale," in *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems*, ser. ICDCS '17, Jun. 2017, pp. 1575–1585.
- [14] C. Stathakopoulou, M. Pavlovic, and M. Vukolić, "State machine replication scalability made simple," in *Proceedings of the 17th European Conference on Computer Systems*, ser. EuroSys '22, Apr. 2022, pp. 17–33.
- [15] G. Yu, X. Wang, K. Yu, W. Ni, J. A. Zhang, and R. P. Liu, "Survey: Sharding in blockchains," *IEEE Access*, vol. 8, pp. 14 155–14 181, 2020.
- [16] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 12, no. 3, pp. 463–492, 1990.
- [17] F. P. Junqueira, B. C. Reed, and M. Serafini, "Zab: High-performance broadcast for primary-backup systems," in *Proceedings of the 41st IEEE/IFIP International Conference on Dependable Systems & Networks*, ser. DSN '11. IEEE, Jun. 2011, pp. 245–256.
- [18] L. Hoang Le, E. Fynn, M. Eslahi-Kelozazi, R. Soulé, and F. Pedone, "DynaStar: Optimized dynamic partitioning for scalable state machine replication," in *Proceedings of the 39th IEEE International Conference on Distributed Computing Systems*, ser. ICDCS '19, Jul. 2019, pp. 1453–1465.
- [19] A. Adya, D. Myers, J. Howell, J. Elson, C. Meek, V. Khemani, S. Fulger, P. Gu, L. Bhuvanagiri, J. Hunter, R. Peon, L. Kai, A. Shraer, A. Merchant, and K. Lev-Ari, "Slicer: Auto-Sharding for datacenter applications," in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, ser. OSDI '16. Savannah, GA, USA: USENIX Association, Nov. 2016, pp. 739–753. [Online]. Available: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/adya>
- [20] K. Lev-Ari, E. Bortnikov, I. Keidar, and A. Shraer, "Modular composition of coordination services," in *Proceedings of the 2016 USENIX Annual Technical Conference*, ser. USENIX ATC '16. USENIX Association, Jun. 2016, pp. 251–264.
- [21] A. Katsarakis, V. Gavrielatos, M. S. Katebzadeh, A. Joshi, A. Dragojevic, B. Grot, and V. Nagarajan, "Hermes: A fast, fault-tolerant and linearizable replication protocol," in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20, Mar. 2020, pp. 201–217.
- [22] K. Lev-Ari, E. Bortnikov, I. Keidar, and A. Shraer, "Composing ordered sequential consistency," *Information Processing Letters*, vol. 123, pp. 47–50, 2017. [Online]. Available: <https://doi.org/10.1016/j.ipl.2017.03.004>
- [23] P. Memarzia, H. Zhang, K. Ho, R. Grosman, and J. Wang, "GaussDB-Global: A geographically distributed database system," in *Proceedings of the 40th IEEE International Conference on Data Engineering*, ser. ICDE '24. IEEE, May 2024, pp. 5111–5118.
- [24] S. Gupta, J. Hellings, and M. Sadoghi, "RCC: Resilient concurrent consensus for high-throughput secure transaction processing," in *Proceedings of the 37th IEEE International Conference on Data Engineering*, ser. ICDE '21. IEEE, Apr. 2021, pp. 1392–1403.
- [25] Z. Milosevic, M. Biely, and A. Schiper, "Bounded delay in Byzantine-tolerant state machine replication," in *Proceedings of the 32nd IEEE International Symposium on Reliable Distributed Systems*, ser. SRDS '13. IEEE, Oct. 2013, pp. 61–70.

Appendix: Artifact Description

Artifact Description (AD)

I. OVERVIEW OF CONTRIBUTIONS AND ARTIFACTS

A. Paper's Main Contributions

- C_1 We propose **Skiff**, the first consensus protocol that uses workload as an electing factor, and provides automatic load balancing functionality.
- C_2 We introduce the **Shipyards** approach, a multi-leader protocol that unifies and manages multiple Skiff instances to eliminate the single-leader bottleneck. Shipyards allows requests to be balanced and ordered based on a sharded key space. It also decouples fault-tolerance from request ordering, providing more robust consistency.
- C_3 We present a comprehensive evaluation of our approach, comparing to state-of-the-art methods.

B. Computational Artifacts

A_1 <https://doi.org/10.5281/zenodo.18717515>

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_2, C_3	Shipyards implementation, Table 1, Figures 9-14

II. ARTIFACT IDENTIFICATION

A. Computational Artifact A_1

Relation To Contributions

The Shipyards code (A_1), written in Golang v1.22, implements the design of Skiff (C_1) along with the multi-leader Shipyards protocol (C_2), providing a working system that realizes the workload-balancing consensus described in the paper. It also includes scripts to reproduce the experimental evaluation (C_3), including comparisons to baseline protocols such as EPaxos, Mencius, Raft, and Multi-Raft (the classic Raft protocol with multiple instances).

Expected Results

Shipyards should achieve workload balancing and eliminate the single-leader bottleneck, while the integrated consensus protocols produce results comparable to or better than state-of-the-art baselines such as EPaxos, Mencius, Raft, and Multi-Raft.

Expected Reproduction Time (in Minutes)

Artifact Setup:

- 1) Installation of AWS EC2 instances and deployment of the base Golang environment (~2 hours)
- 2) Building the gateway, routing tables, network configuration, and initial testing (~2-4 hours) Each experimental round takes approximately 100 seconds, a full execution of all experiments is estimated to take 1 day.

Artifact Execution: Each experimental round takes ~100 seconds. A full execution of all experiments takes roughly a full day.

Artifact Analysis: Collect logs, aggregate results, calculate throughput, and generate figures. This process takes ~1-2 days.

Artifact Setup (incl. Inputs)

Hardware:

- **Cloud platform:** Amazon AWS EC2 (<https://aws.amazon.com/ec2/>)
- **Instance type:** t3.micro, 2 vCPUs, 1 GiB memory, 1 × 8 GiB gp3 storage
- **Number of instances:** 10 (5 servers, 5 clients)
- **Instance regions:** US East (N. Virginia), Asia Pacific (Tokyo), Europe (Paris)
- **Operating system:** Ubuntu 22.04 LTS
- **Network:** Cross-region connectivity via AWS VPC

Software:

- **Shipyards code:** Main computational artifact (<https://github.com/xyang76/shipyards>)
- **Golang:** 1.22 (<https://golang.org/>)
- **Baseline protocols:** EPaxos and Mencius (<https://github.com/efficient/epaxos>), reference Raft implementation (<https://github.com/elib/raft>)

Datasets / Inputs: All inputs required by the artifact are generated dynamically during execution. Clients generate request workloads according to the parameters specified in the provided scripts, and the servers process these requests to produce the experimental results. No external datasets are required. Instructions for configuring workload parameters and running the experiments are included in the Shipyards repository (<https://github.com/xyang76/shipyards>).

Installation and Deployment: The artifact requires Golang 1.22 (<https://golang.org/>). To deploy the experiments, launch the required number of EC2 instances (as specified in the Hardware section) and clone the Shipyards repository to the respective nodes. To run the experiments, start the components as follows (detailed execution instructions are available in the GitHub repository):

- **Master node:** Obtain peer IDs and IP addresses using `go run main.go -r 1` or start/stop using `./master.sh` / `./kmaster.sh`.
- **Servers:** `go run main.go -r 2 -maddr [master address] -a [approach type]`
- **Clients:** `go run main.go -r 3 -maddr [master address] -a [approach type] -w [write percent]`

Artifact Execution

The experiment workflow consists of four main tasks:

- T_1 (**Master Initialization**): Start the master node.

- T_2 (**Server Deployment**): Launch all server instances. Servers wait to accept client requests.
- T_3 (**Client Execution**): Launch clients to generate request workloads according to configured parameters:
 - w = write percentage
 - rnd = total rounds
 - q = requests per round
 - mb = max batch size

The dependencies between tasks are: $T_1 \rightarrow T_2 \rightarrow T_3$.

Artifact Analysis (incl. Outputs)

The Shipyard artifact produces logs of all client requests and server responses during execution. Each log entry contains timestamps and successfully executed commands.

These logs capture the system behavior and can be used to reproduce the evaluation metrics reported in the paper, such as throughput and workload distribution.